

Unix Lab # 2 - Unix File permissions, process control, pipes

Submitted by: Tomer Benveniste – 207961954 and Carmi Frank - 206463846

1. Describe 3 different ways of setting the permission of a file/Directory to

`r--r--r--`

Here are three different ways to set a file or directory's permissions to `r--r--r--` (read-only for everyone):

- **Numeric Mode:** `chmod 444 <filename>`.
4 represents 'read' permission applied to the Owner, Group, and Others (the vector [r,w,x] can be represented as a bit vector, which in this case 4 is [1,0,0] which is actually [r,-,-]).
- **Symbolic Mode (All Users):** `chmod a=r <filename>`.
(`'a'` stands for all users, and `'='` strictly sets their permission to 'read').
- **Symbolic Mode (Explicit Categories):** `chmod u=r, g=r, o=r <filename>`.
(Individually sets 'read' for the User/owner, Group, and Others).

```
tomerbenb13@ubuntu:~$ touch test_file.txt
tomerbenb13@ubuntu:~$ ls -l test_file.txt
-rw-rw-r-- 1 tomerbenb13 tomerbenb13 0 Apr  6 10:47 test_file.txt
tomerbenb13@ubuntu:~$ chmod 444 test_file.txt
tomerbenb13@ubuntu:~$ ls -l test_file.txt
-r--r--r-- 1 tomerbenb13 tomerbenb13 0 Apr  6 10:47 test_file.txt
tomerbenb13@ubuntu:~$ touch test_file_2.txt
tomerbenb13@ubuntu:~$ ls -l test_file_2.txt
-rw-rw-r-- 1 tomerbenb13 tomerbenb13 0 Apr  6 10:48 test_file_2.txt
tomerbenb13@ubuntu:~$ chmod a=r test_file_2.txt
tomerbenb13@ubuntu:~$ ls -l test_file_2.txt
-r--r--r-- 1 tomerbenb13 tomerbenb13 0 Apr  6 10:48 test_file_2.txt
```

```
tomerbenb13@ubuntu:~$ touch test3.txt
tomerbenb13@ubuntu:~$ ls -l test3.txt
-rw-rw-r-- 1 tomerbenb13 tomerbenb13 0 Apr  6 10:51 test3.txt
```

```
tomerbenb13@ubuntu:~$ chmod u=r,g=r,o=r test3.txt
tomerbenb13@ubuntu:~$ ls -l test3.txt
-r--r--r-- 1 tomerbenb13 tomerbenb13 0 Apr  6 10:51 test3.txt
```

2. create a folder in your desktop name it lab3, make it completely private (all permissions for the owner only, add 3 files into it using touch command, what are the permissions given to the files? *[Write the command you used and the output for showing the files permissions]*).

```
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~$ mkdir lab3
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~$ chmod 700 lab3
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~$ cd lab3
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~/lab3$ touch file1 file2 file3
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~/lab3$ ls -l
total 0
-rw-rw-r-- 1 carmi carmi 0 Apr  4 16:51 file1
-rw-rw-r-- 1 carmi carmi 0 Apr  4 16:51 file2
-rw-rw-r-- 1 carmi carmi 0 Apr  4 16:51 file3
```

The folder is completely private because we used “chmod” (“Chang mode”), which is the command to manage file and directory permissions. The command sets to how the permissions are given and what can they do :

- User - The owner of the folder
- Group - Every file in Linux has one group, the group members share the same access rights.
- Others - Everyone else on the system.

The number “700” sets the rights as each digit is a sum of numbers represents the following rights:

- Write - 4
- Read - 2
- Execute - 1

Therefore, the folder “lab3” is completely private. The user (owner) can write, read and execute but the rest of users can not.

touch - This command was used to create the three empty files simultaneously.

The permissions are “-rw-rw-r-”. Means the owner and group can write and read while other users can only read. We can notice that access rights are not inherited in Linux by default.

3. Use find to display the names of all files in the system that are bigger than 1MB*[write the command you used]*.

We will use the command: **find / -type f -size +1M**

- **find:** The command used to search for files and directories based on specific criteria.
- **/:** This tells the system to start searching from the root directory, meaning it will scan the entire system.
- **-type f:** This filter restricts the search results to standard files only (ignoring directories, symbolic links, etc.).
- **-size +1M:** This specifies the size condition. The + means "greater than," and 1M stands for 1 Megabyte.

- **2>/dev/null**: This hides error messages from the terminal output, keeping the screen clean of 'permission denied' messages. **2** - Refers to "Standard Error" (stderr), the specific stream where the system sends error messages, **>** - The redirect operator, **/dev/null** - A special system file that instantly deletes any data sent to it..

```

tomerb13@ubuntu:~$ find / -type f -size +1M 2>/dev/null
/usr/src/linux-headers-6.17.0-14-generic/tools/objtool/objtool-in.o
/usr/src/linux-headers-6.17.0-14-generic/tools/bpf/resolve_btfids/resolve_btfids
/usr/src/linux-headers-6.17.0-14-generic/tools/bpf/resolve_btfids/libbpf/libbpf.a
/usr/src/linux-headers-6.17.0-14-generic/tools/bpf/resolve_btfids/libbpf/staticobjs/libbpf.o
/usr/src/linux-headers-6.17.0-14-generic/tools/bpf/resolve_btfids/libbpf/staticobjs/libbpf-in.o
/usr/src/linux-headers-6.17.0-14-generic/Module.symvers
/usr/libexec/gnome-initial-setup
/usr/libexec/gnome-shell-perf-helper
/usr/libexec/bluetooth/bluetoothd
/usr/libexec/gcc/x86_64-linux-gnu/13/cc1
/usr/lib/udev/hwdb.d/20-usb-vendor-model.hwdb
/usr/lib/udev/hwdb.d/20-pci-vendor-model.hwdb
/usr/lib/udev/hwdb.d/20-OUI.hwdb
/usr/lib/udev/hwdb.bin
/usr/lib/x86_64-linux-gnu/libvulkan_lvp.so
/usr/lib/x86_64-linux-gnu/libmvec.a
/usr/lib/x86_64-linux-gnu/libjavascriptcoregtk-6.0.so.1.6.9
/usr/lib/x86_64-linux-gnu/espeak-ng-data/ru_dict
/usr/lib/x86_64-linux-gnu/espeak-ng-data/cmn_dict
/usr/lib/x86_64-linux-gnu/libclang-18.so.18

```

4. Use `grep` to isolate the line in `/etc/passwd` that contains "ubu". *[Write the command you used].*

```

carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~$ grep "ubu" /etc/passwd
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~$ ^C
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~$ grep "carmi" /etc/passwd
carmi:x:1000:1000:Carmi Frank,,,:/home/carmi:/bin/bash
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~$ S

```

"grep" - Stands for **G**lobal **R**egular **E**xpression **P**rint, and it is a command which inputs an expression like "ubu" and searches it inside the path provided. In our case "/etc/passwd".

We can see, in the screenshot provided, that "ubu" hasn't been found inside the specified path. We note that "/etc/passwd" is a path to a file containing essential information for every user account on the system. The command outputs nothing because there is no user with the expression "ubu" in the system. On the other hand, "carmi" returns an output because there is such a user.

5. Use `find` and `grep` and `sort` to display a sorted list of all files in the `/home` subdirectory tree that contain the word `hello` somewhere inside them. *[Write the command you used, test it by creating files with the needed content].*

The command is: `find /home -type f -exec grep -l "hello" {} + 2>/dev/null | sort`

- `find /home -type f`: Searches the '/home' directory for standard files.

- **-exec grep -l "hello" {} +** : Searches inside those files for the word "hello" and outputs only the filenames (-l).
- **2>/dev/null**: Hides the irrelevant "Permission denied" errors.
- **| sort**: Takes that final list of filenames and sorts them alphabetically.

```
tomerbenb13@ubuntu:~$ echo "hello world" > ~/b_match.txt
tomerbenb13@ubuntu:~$ echo "no match here" > ~/a_fail.txt
tomerbenb13@ubuntu:~$ echo "hello again" > ~/c_match.txt
tomerbenb13@ubuntu:~$ find /home -type f -exec grep -l "hello" {} + 2>/dev/null | sort
/home/tomerbenb13/.bash_history
/home/tomerbenb13/b_match.txt
/home/tomerbenb13/.cache/gstreamer-1.0/registry.x86_64.bin
/home/tomerbenb13/.cache/tracker3/files/http%3A%2F%2Ftracker.api.gnome.org%2Fontology%2Fv3%2Ftracker%23Documents.db-wal
/home/tomerbenb13/c_match.txt
/home/tomerbenb13/play/hello.txt
/home/tomerbenb13/play/lab2.txt
tomerbenb13@ubuntu:~$
```

6. Create a file containing some lines that you think would match the regular expression: `(^[0-9]..[a-zA-Z]+$)` and some lines that you think would not match. Use `grep` to see if your intuition is correct. *[write the command you*

```
carmifr@DESKTOP-81VCQE2:~/play$ cat file
match:
111xcvfr
2wwfdgfg
3--rrtrt
4bbrrt
5%%vsdf

not match:
11sdfsef-
r4df87368
4444asfs
1rrrr1

carmifr@DESKTOP-81VCQE2:~/play$ grep -E '^[0-9]..[a-zA-Z]+$' file
111xcvfr
2wwfdgfg
3--rrtrt
4bbrrt
5%%vsdf
```

We used “nano” to create a file named “file” and added strings we thought would match the rules of “`grep -E '^[0-9]..[a-zA-Z]+$' test.txt`”. The rules are the following:

1. “`^[0-9]`” - means the line must start with one, and only one digit.
2. “`..`” - gap of two characters. The character types do not matter but have to be only 2.
3. “`[a-zA-Z]+`” - there must be letters next, no matter how much
4. “`$`” - The line must end after the letters. no other characters allowed.

We used “cat” to show the file content.

We run “grep” to find the expressions that match the rules specified.

7. Create several files which their names would match the regular expression: `(^[0-9]..[a-zA-Z]+$)` and some files names that you think would not match. Use `ls` and `grep` to list all files that match the regular expression. *[write the command you used to find it out into your report]*

The command is: `ls | grep -E '^[0-9]..[a-zA-Z]+$'`

- `ls`: Lists all files in the current directory.
- `| grep -E`: Pipes that list into 'grep' using Extended Regular Expressions.
- `'^[0-9]..[a-zA-Z]+$'`: Requires the filename to start with exactly one digit (`^[0-9]`), followed by exactly two characters of any kind (`..`), and end with one or more letters (`[a-zA-Z]+$`).

```
tomerbenb13@ubuntu:~$ touch 1__abc 5xyZ
tomerbenb13@ubuntu:~$ touch a__abc 12345 1ab
tomerbenb13@ubuntu:~$ ls | grep -E '^[0-9]..[a-zA-Z]+$'
1__abc
5xyZ
```

8. run the command `sleep 1000` in the background, try to stop it in the same terminal *[write the commands you used to find it out into your report]*

```
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~$ sleep 1000 &
[1] 9333
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~$ jobs
[1]+  Running                  sleep 1000 &
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~$ kill %1
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~$ jobs
[1]+  Terminated              sleep 1000
carmi@carmi-VivoBook-ASUSLaptop-TP420UA-TM420UA:~$
```

“sleep” - this command tells Linux to wait 1000sec.

“&” - tell Linux run the previous command in the background

“[1] 9333” - is the output. It reports the user the process ID (9333), which is a unique number on the OS used to track this task, and the job number of the specific terminal ([1]).

“jobs” - lists all processes that were started by and are currently managed by this specific terminal.

“kill %1” - kill process %[process id].

9. Try to create an orphan process, by killing the parent process of a background process `sleep 1000`. Did you succeed? What do you understand of it? *[write the commands you used to find it out into your report]*

We use a temporary sub-shell to test this safely without closing our main terminal.
The commands:

- **bash**: opens a temporary sub-shell.
- **sleep 1000 &**: starts the process in the background.
- **ps -f**: views the processes to see sleep's original Parent PID (PPID).
- **kill -9 \$\$**: kills the parent (the current temporary bash shell)
- **ps -ef | grep "sleep 1000"**: back in your main terminal, check on the sleep process.

The screenshot is down below:

```
tomerbenb13@ubuntu:~$ bash
tomerbenb13@ubuntu:~$ sleep 1000 &
[1] 5202
tomerbenb13@ubuntu:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
tomerbe+    3804    3797  0 13:58 pts/0        00:00:00 bash
tomerbe+    5196    3804  0 16:02 pts/0        00:00:00 bash
tomerbe+    5202    5196  0 16:02 pts/0        00:00:00 sleep 1000
tomerbe+    5203    5196  0 16:02 pts/0        00:00:00 ps -f
tomerbenb13@ubuntu:~$ kill -9 $$
Killed
tomerbenb13@ubuntu:~$ ps -ef | grep "sleep 1000"
tomerbe+    5202    2755  0 16:02 pts/0        00:00:00 sleep 1000
tomerbe+    5205    3804  0 16:03 pts/0        00:00:00 grep --color=auto sleep 1000
tomerbenb13@ubuntu:~$
```

Explanation: An orphan process is created when a parent process is terminated while its child is still running. In the screenshot, the **sleep 1000** process originally belonged to the temporary **bash** shell (Parent PID **5196**). When we forcefully killed that parent shell, the **sleep** process was not destroyed. Instead, it was left orphaned and was automatically adopted by a background system manager, which changed its Parent PID to **2755**.

10. Write a script called myid.sh that receives the id number of the student and prints out if it is odd or even in this format:
The id number is:xxxxxxxxx
This id is (Even\Odd)

```
carmi@DESKTOP-81VCQE2:~/play$ nano ex2_sec10
carmi@DESKTOP-81VCQE2:~/play$ chmod +x ex2_sec10
carmi@DESKTOP-81VCQE2:~/play$ ./ex2_sec10 206463846
ID number is:206463846
This id is Even
carmi@DESKTOP-81VCQE2:~/play$ ./ex2_sec10 206463841
ID number is:206463841
This id is Odd
carmi@DESKTOP-81VCQE2:~/play$
```

We used “nano” to create the file, “chmod” to make it executable and “.” to execute it. Below is the script file content:

```
#!/bin/bash
ID=$1
echo "ID number is:$ID"
if [  $$(ID \% 2)$  -eq 0 ]; then
    echo "This id is Even"
else
    echo "This id is Odd"
fi
```

#!/bin/bash - tells linux it is about to be executable

ID=\$1 - Tell Linux, the first string passed to terminal after exe command will enter the variable ID.

echo prints the expression inside “.”. \$ID is being recognized by Linux and replaced in the actual content of ID (the id we entered).

if - normal condition\

$$(...)$ - Arithmetic expression, tell Bash to do math on the expression. %- divide in 2 and output the remainder.

-eq - comparison operator.

0 - the remainder for even number

then - marks the start of the code that will run if the math was correct

else - if remainder is not 0 then print “This is Odd id”

fi - end the if.